

VarBlockSpMM: Shape-Aware CUDA Execution for Packed Variable-Block Sparse-Dense Products

Cohen Brunner Thomas
Independent Research Project

6 September 2026

Abstract

VarBlockSpMM is a standalone CUDA implementation of FP32 sparse-dense products for matrices stored as packed dense blocks with variable sizes. It avoids scalar expansion, fixed-block tiling, output atomics, and execution-time temporary workspace. On one RTX 5060 Ti, a full-core follow-up on three additional matrix seeds gives geometric fastest-tested-library-to-direct time ratios of 1.139 for individual products and 1.132 for batches of eight queued products. The core contains 128 configurations spanning four panel widths, four shape distributions, two locality patterns, and four degrees. The kernel shares dense input tiles across rows and reuses sparse values across RHS accumulators. Matched comparisons show that its narrow-panel policy gives speedup factors of 1.777 at width 8 and 1.771 at width 16 over the previous policy. On the generated native component, the prepared-host startup ratio is 3.953 including one completed product, while the steady-state BSR8 ratio is 0.390 and favors BSR8. Published-matrix and dense-library controls identify cases better served by other representations. Verified build records and raw measurements support separate comparisons of execution, startup, and storage, defining the conditions under which retaining packed variable blocks is useful.

1 Introduction

Applications with dense variable blocks face a representation choice when using sparse GPU libraries. Scalar CSR expands block metadata to individual entries. Fixed-block BSR subdivides the original blocks and may require a new value layout. Grouped GEMM can retain block shapes, but overlapping contributions require separate calls or a reduction. VarBlockSpMM executes the packed product directly, so an application can retain its block layout and avoid these conversion steps.

This is useful when direct execution is faster, when scalar metadata is costly, or when a matrix is used too few times to repay library conversion and setup. We measure these benefits separately. The intended input has dense stored blocks. Artificially grouping scattered scalar nonzeros can instead waste storage and arithmetic, as the published-matrix controls show.

We study $C = AB$, where $A \in \mathbb{R}^{M \times K}$ is a packed variable-block sparse row matrix and $B \in \mathbb{R}^{K \times N}$ and $C \in \mathbb{R}^{M \times N}$ are column-major dense panels. The supported block dimensions are 8, 16, $\dots$, 64, with independently chosen row and column partitions. Dense panel widths are $N \in \{8, 16, 32, 64\}$. The implementation uses FP32 values and accumulation for non-transpose multiplication with $\alpha = 1$ and $\beta = 0$.

The implementation specializes execution by panel width while retaining runtime block dimensions. One CTA owns each block row, shares the dense input tile, and writes the completed output once. This combines a small execution plan with direct access to the application’s packed values.

The contribution is a standalone execution policy for this packed format, with an experimentally measured boundary against conversion-based library alternatives. It combines full-panel accumulation for narrow RHS widths with shape-dependent staging for wider panels, retaining one writer per output and requiring no execution workspace. The matched experiments test which choices improve this implementation. The library comparisons test when their combined benefit exceeds the benefit of padding-free BSR8 subdivision or scalar CSR expansion. These are implementation and measurement contributions. They do not establish a new sparse storage model or an end-to-end application speedup.

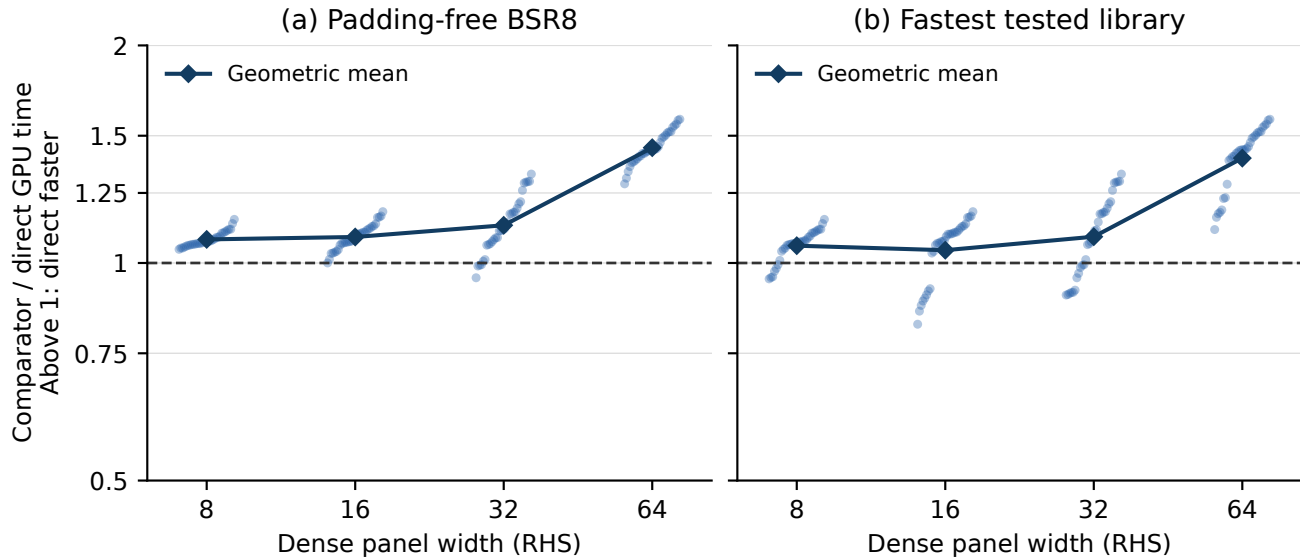


Figure 1: Direct execution by panel width on the full 128-case synthetic core. Each point is one configuration averaged over three processes. Lines show geometric means across all 32 configurations at each width. The right panel uses the fastest measured library method within each process, including BSR32 on uniform blocks. Horizontal offsets separate points and have no numerical meaning. The vertical axis is logarithmic. Ratios above one favor VarBlockSpMM.

Startup and storage measurements separately test the cost of adopting each representation, with raw measurements tied to verified builds. Figure 1 shows where the tested widths and workloads favor direct execution.

2 Related work

Variable blocking is established prior art. Vuduc and Moon studied variable and unaligned blocks for sparse matrix-vector multiplication [6]. OSKI defines variable block row storage through independent logical row and column partitions [7]. VarBlockSpMM uses this structural model for a dense multi-column right operand on CUDA.

GPU SpMM work has examined row assignment, load balance, instruction-level parallelism, and reuse. Yang, Buluç, and Owens describe row-splitting and merge-based approaches [1]. GE-SpMM caches sparse rows and merges warps to reduce redundant loads [8]. Tensor-core methods such as Okanovic et al. trade packing and preprocessing for dense matrix-instruction throughput [2]. These techniques motivate the present kernel, but this study does not establish superiority over compatible research implementations.

The motivating application is adaptive construction of hierarchical H^2 matrices. Boukaram et al. describe a non-uniform batched BSR product implemented through non-uniform batched GEMM operations [5]. The present work addresses the underlying packed-block product. We add a generated covariance near-field component whose partitions come from a spatial cluster tree. This tests a concrete dense-block component, while excluding far-field compression and the complete construction pipeline.

NVIDIA provides scalar and fixed-block SpMM through cuSPARSE [3], and per-group GEMM dimensions through grouped cuBLAS [4]. These form the library controls. Asynchronous global-to-shared copies can overlap tile movement and arithmetic [9]. These mechanisms are established techniques. We combine and evaluate them for this particular variable-block operation.

3 Representation and kernel

Let block-row boundaries be $r_0 < \dots < r_R$, and block-column boundaries be $c_0 < \dots < c_Q$. Block (i, j) has height $h_i = r_{i+1} - r_i$ and width $w_j = c_{j+1} - c_j$. Its column-major values are packed without rounding its dimensions to a global block size, and its contribution is

$$C[r_i : r_{i+1}, :] += A_{ij}B[c_j : c_{j+1}, :].$$

Block-row pointers and column indices locate the blocks. Scalar offsets and value offsets are 64-bit. Block indices remain 32-bit. Host validation checks monotonic offsets, dimensions, index bounds, and payload sizes. A matrix owns its GPU storage and row classifications. Execution plans hold non-owning views, so the matrix must outlive its plans.

3.1 Output ownership and accumulation

One cooperative thread array, or CTA, owns one block row at every supported RHS width. Threads outside the actual row height do no output work. Each active thread accumulates all contributions to its outputs and writes them once. CTAs never update the same scalar, so execution needs neither output atomics nor partial-output reduction storage. Work imbalance between rows can still limit performance.

RHS 8 and 16 use 64-thread CTAs, with each active thread owning all RHS columns of one scalar row. RHS 32 assigns eight output columns per thread, and RHS 64 assigns sixteen. Multiple accumulators let each loaded sparse value feed independent multiply-add operations. Their register cost must be balanced against the number of resident CTAs and active threads.

Ownership and tile sequence for one block row

Ownership	One CTA writes the entire output panel of block row i .
Initialization	Each thread sets its owned accumulators to zero.
First tile	Load the first reusable B tile into shared memory.
Pipeline step	Initiate the next tile copy into the alternate buffer, then compute the current block contribution.
Boundary	Wait for copies and synchronize the CTA before consuming the next tile or reusing a buffer.
Completion	Write each owned output once, including zero for an empty row.

Figure 2: The tall-row RHS-32 algorithm with its alternating two-buffer schedule. All supported widths retain disjoint output ownership.

3.2 Full-panel accumulation for RHS 8 and 16

The narrow-panel kernel launches one 64-thread CTA per block row. Thread $t < h_i$ owns $C[r_i + t, 0 : N]$, with N FP32 accumulators. For each stored block, all 64 threads cooperatively load its $w_j \times N$ input tile into shared memory. A barrier makes that tile visible before use. Each active thread then loads one sparse value at a time and reuses it across all N columns. A second barrier protects shared-memory reuse when another block follows.

The shared leading dimension is 65 floats. The RHS-8 and RHS-16 kernels allocate 2,080 and 4,160 shared-memory bytes per CTA, respectively. Since the largest supported row has 64 scalars, one CTA covers the entire panel without a row-size threshold or a second output tile. Empty rows write zero.

The narrow policy was selected on development seed 4 at 256, 1,024, and 4,096 block rows, degrees 2 and 16, and all four shape distributions. We compared staged kernels with different thread and accumulator counts and warp-reduction candidates, checking every output against a full CPU reference. The final policy uses full-panel accumulation and staging at both widths. Evaluation seeds are separate from this development seed. The matched campaign also measures the previous narrow policy in the same executable and process, so its improvement does not depend on comparing archived times.

3.3 Staging and dispatch

At every supported width, each local output row consumes the same dense input tile for a stored block. Cooperative staging loads this tile once per CTA and reuses it across local rows. A shared leading dimension of 65 floats reduces power-of-two bank-alignment conflicts. The synchronous RHS-32 and RHS-64 kernels use 8,320 and 16,640 shared-memory bytes per CTA, respectively. The narrow kernels use the smaller CTAs described above. RHS 32 and 64 retain their width-specific staging and buffering policies.

For RHS 32, rows of at most 16 scalars use a 128-thread single-buffer CTA. Taller rows use a 256-thread CTA with two tiles, consuming 16,640 shared-memory bytes. The next input tile is prefetched before arithmetic on the current tile. Pipeline waits and CTA barriers protect visibility and buffer reuse. A mixed-height matrix is split into two launches only when its mean block degree is at least eight. Homogeneous matrices need only the applicable launch. At lower mean degree, the mixed matrix uses the unsplit policy.

Double buffering is not enabled for RHS 64. A development experiment increased shared allocation to about 32.5 KiB per CTA and regressed a representative median from 2.891 to 4.419 ms. This observation motivated a width-specific policy rather than a universal pipeline. All dispatch thresholds were selected from development sweeps and held fixed for the main library comparison.

4 Experimental design

The main campaign measures the optimized dispatch described above against persistent library controls. All synthetic and application measurements were collected with the current implementation. The separate kernel ablations compare direct implementations under their stated protocols.

4.1 Persistent library controls

Scalar CSR. Synthetic dense blocks are expanded into scalar CSR once. Plans retain arrays, descriptors, handles, and workspace. We measure the default algorithm, explicit CSR algorithm 1 with preprocessing, algorithm 2, and algorithm 3 with preprocessing. All use FP32 compute, column-major panels, and the same non-transpose operation. Preprocessing and initial descriptor creation occur during setup or warmup, outside timing. NVIDIA documents preprocessing benefits for algorithms 1 and 3 [3].

The *CSR lower envelope* is the minimum GPU median among those four configurations within each process. Selecting on measured data favors this comparator. The envelope has no deployed tuning policy or charged tuning cost. It lets us check whether a direct-method win depends on the default library algorithm. Host ratios use a separate minimum over host medians, so they need not select the same algorithm.

Grouped cuBLAS. Blocks with equal height and width are grouped within each ordinal block slot. A slot contains at most one block from each row, so its GEMM outputs do not overlap. Slots execute sequentially. The first uses $\beta = 0$, and subsequent slots use $\beta = 1$. The handle, packed values, shape groups, and device pointer arrays persist.

The cached control refreshes input and output pointer arrays only when their base addresses change. It omits the full output clear when every row is nonempty. For matrices containing empty rows it retains the clear to initialize those rows. A separate changing-address experiment alternates between two allocated input/output pairs and includes pointer refreshes. It is paired with a direct method that alternates the same pairs. A stream-ordered CUDA kernel computes the input and output pointer arrays from persistent device offsets when needed. This avoids relying on the lifetime or staging behavior of temporary pageable host arrays. The first measured call reuses the final warmup addresses. The remaining 19 calls alternate addresses and refresh the grouped pointers. Changing-address timing includes these pointer-generation launches. The slot decomposition still incurs sequential GEMM calls and repeated output traffic, so it is a specific library composition rather than an oracle over possible grouped algorithms.

Fixed-block BSR. The uniform-32 synthetic subset is also measured with cuSPARSE BSR algorithm 1. Block values are converted to the required row-major block layout outside timing, while dense panels remain column-major. This gives a format-matched control where variable blocking is unnecessary.

Subdivided BSR8. Every supported dimension is divisible by eight. We therefore also subdivide each stored variable block into 8-by-8 blocks and measure cuSPARSE BSR algorithm 1 on every input. The scalar matrix and stored scalar count are unchanged, and no padding is introduced. A block of shape $h \times w$ becomes $(h/8)(w/8)$ fixed blocks, increasing block metadata while exposing a different scheduling and reuse tradeoff. Conversion to row-major BSR tiles occurs outside steady-state timing. The published scalar inputs retain their internal zeros in this control, whereas compact CSR removes them. BSR8 measures the effect of subdivision, while compact CSR measures the benefit of removing internal zeros.

4.2 Synthetic workloads

The core grid contains 128 configurations with 4,096 block rows and columns, degrees 2, 4, 8, 16, four panel widths, four size distributions, and two locality patterns. Uniform uses size 32. Low variance samples $\{24, 32, 40\}$, high variance samples $\{8, 16, \dots, 64\}$, and bimodal samples $\{8, 16, 48, 64\}$. Row and column dimensions are drawn independently. Every row in the core grid has exactly the requested degree. Columns are sampled without replacement and sorted. Random locality spans all block columns. Local locality samples cyclic offsets within plus or minus the requested degree. Stored values are uniform on $[-1, 1]$.

The extension adds 40 configurations. It evaluates 256, 1,024, and 4,096 block rows and columns for high-variance and bimodal sizes, random locality, all panel widths, and either constant degree 16 or the repeating degree pattern 0, 1, 16, 16. The latter has mean degree 8.25 and explicitly includes empty rows. It is constructed by retaining zero blocks, one block, or all blocks from successive rows of the degree-16 parent matrix. Eight constant-degree 4,096-row configurations already belong to the core grid. Thus the union contains 168 unique configurations.

The synthetic campaign holds matrix seed 1 fixed and repeats each configuration in three fresh processes. The dense input contains independent pseudorandom uniform values on $[-1, 1]$, generated with seed 9123. Changing-address buffers contain identical values to isolate address-dependent work.

4.3 Published matrices and partitioning

We also test BCSSTK13, BCSSTK14, and BCSSTK15 from the Harwell-Boeing collection, downloaded through SuiteSparse [10]. They originate from fluid-flow eigenvalues, the Omni Coliseum roof, and an offshore platform module, respectively [11]. Their original orders are 2,003, 1,806, and 3,948.

Symmetric Matrix Market input is expanded, duplicates are combined, and explicit zeros are removed. Values are divided by the maximum absolute entry and converted to FP32. The rows and columns are padded only at the outer boundary to the next multiple of eight. In the original scalar ordering, row partitions cycle through 8, 16, 32, 64 and column partitions through 64, 32, 16, 8. The final partition is truncated to the remaining multiple of eight. A block is stored if it contains a source nonzero. Other entries inside that block are represented as zeros.

This deterministic procedure preserves application-derived values and sparsity after normalization, but uses artificial partitions in place of native finite-element or hierarchical partitions. The measured compact-CSR controls retain only source nonzeros in the padded scalar domain, using explicit algorithms 1, 2, and 3 with preprocessing for 1 and 3. Their lower envelope is computed as above. Cached grouped cuBLAS uses the identical packed blocks as the direct method. This separates the cost of block fill from scalar sparse execution. All four panel widths and three fresh processes are retained. The partitions have 68, 61, and 132 block rows, respectively. RHS 8 and 16 use the same 64-thread staged kernel as the synthetic inputs.

4.4 Geometry-derived covariance near field

We construct the dense near-field component of a spatial covariance operator, following the bounding-box admissibility model described by Boukaram et al. [5]. Unlike grouping a pre-existing scalar sparse matrix, the spatial tree defines the dense blocks before matrix assembly.

Each input contains 2,048 or 8,192 points in three dimensions. Uniform geometry samples the unit cube. Clustered geometry places three quarters of the points in a clipped normal cloud with coordinate mean 0.3 and standard deviation 0.07, and leaves the remaining quarter uniform. Independent NumPy generator seeds 1, 2, and 3 give twelve matrices. For each matrix and panel width, three fresh processes repeat the same instance, giving 144 process observations.

A binary spatial tree splits along the longest bounding-box axis, using a stable coordinate sort and a midpoint rounded to a multiple of eight. A leaf stops at eight points, or at no more than 64 points when its bounding-box diameter is at most 0.18. Eight-point granularity is an explicit restriction of this application model. No points are padded, dropped, or grouped after scalar matrix assembly. The reordered leaf indices define both partitions. Uniform geometry at 2,048 points produces only size-eight leaves and is retained as a useful fixed-block control.

For leaf boxes s, t , let D denote Euclidean box diameter and dist the minimum Euclidean distance between boxes. We retain leaf pairs failing

$$\frac{D(s) + D(t)}{2} \leq 0.5 \text{dist}(s, t).$$

Each retained block stores the Gaussian covariance entries

$$A_{ij} = P^{-1/2} \exp(-\|x_i - x_j\|^2 / (2(0.1)^2)),$$

where P is the point count. The amplitude fixes the numerical scale across sizes. Other blocks are zero in this component. The far-field component of the full covariance operator is outside this experiment. The resulting near-field component need not itself be positive definite.

An independent scalar-row enumeration recomputes the covariance values and provides the compact-CSR CPU reference. Preparation checks every packed block against this reference exactly in FP32. All stored values are nonzero for these inputs. Thus the experiment has full block fill and tests native spatial partitions within this specified model. It is not a production solver trace or an end-to-end hierarchical computation.

Some near-field inputs occupy a large fraction of the scalar matrix. We therefore include a persistent dense cuBLAS SGEMM control on all native and published-matrix inputs. It expands the compact scalar source to a zero-filled column-major dense matrix during startup, retains that matrix and a handle, and uses the same non-transpose operation and default cuBLAS math contract as the grouped control. Its output passes the same full-reference tolerances. Dense expansion is impractical at the main synthetic grid’s scalar dimensions and is not attempted there.

4.5 Matched mechanism experiments

The matched campaign crosses 256 and 1,024 block rows, degrees 2 and 16, high-variance and bimodal shapes, and four RHS widths with random locality. Each of the 32 configurations uses three matrix seeds and three fresh processes per seed, giving 288 processes. Both instance and process variation are therefore present, without treating timing trials as independent matrix samples.

For RHS 8, a complete $2 \times 2 \times 2$ factorial design varies linear versus eight-row output mapping, one versus two accumulators, and one versus two CTAs per block row. Every CTA has 256 threads. For wider panels, a global-load four-accumulator control and the selected wider accumulator count both use one 256-thread CTA per row. Cooperative staging is compared with direct loads using the same output mapping and CTA size. Single and double buffering use the same indirect row order, CTA size, and accumulator count. At RHS 8 and 16, an additional pair compares global and shared input loads with identical 64-thread CTAs and full-panel accumulators. The previous narrow policy and the production dispatch are measured alongside these controls. Each variant is checked against a full double-accumulating CPU reference, rather than only sampled output probes.

These comparisons isolate the named policy choices at fixed input and launch dimensions. Register allocation, instruction scheduling, and active thread count can change as consequences of a choice. They are not held artificially constant. The factorial marginal effects average across the other two RHS-8 factors, and the retained per-variant timings expose interactions rather than assuming that speedups multiply.

4.6 Timing and correctness

Each method receives five warmups and 20 measured executions. Methods are randomly ordered within each process. Configuration and process-label jobs are shuffled before serial execution, with one GPU benchmark active at a time. Synthetic order uses seed 20260905, published-matrix order uses 20260906, and both supplementary campaigns use 20260907. Three process labels identify repeated observations of the same matrix, rather than three different matrix instances.

CUDA events bracket each execute call on the default stream, followed by an end-event synchronization. Synchronized host durations are also retained. The intervals measure execution calls, including any device idle time while the host submits later work. This matters particularly for library compositions with multiple calls, so the intervals do not isolate kernel-instruction time. Allocation, format conversion, initial input transfer, and correctness validation are excluded. Output buffers are poisoned before each method’s warmups, and correctness is checked both after warmup and after measurement. Every raw GPU and host duration, method position, command, executable checksum, and process status is saved. Medians use the mean of the two middle observations for an even sample count.

The revision uses the RTX 5060 Ti with compute capability 12.0, CUDA runtime and driver API 13.4, Windows, MSVC 19.44, Release compilation, native CUDA architecture, and fast FP32 math. The Windows display driver reports version 32.0.16.1656. The GPU is not clock-locked or isolated from all operating-system activity. Per-process environment records accompany the synthetic timings.

The correctness suite covers both localities for every combination of four size distributions, four widths, and degrees 1, 4, 8, 16, giving 128 cases. Sixteen additional cases contain an empty first row. It tests explicit CSR algorithms, BSR32 and BSR8, cached pointer reuse, changed input and output addresses, and non-default streams. Deterministic tests cover all 64 block shapes at every RHS width, mixed empty rows, and completely empty matrices. NaN initialization before every comparison detects missing writes, and a deliberately empty operation must fail the comparison helper. Queued pointer changes are checked without intervening host synchronization. Every CUDA status in the test execution path is checked. Full output is compared with a CPU reference that accumulates the entire dot product in double before one final FP32 cast. Failure thresholds are maximum absolute error 5×10^{-4} or relative ℓ_2 error 5×10^{-5} , with NaN and infinity rejected.

Each timed synthetic matrix also receives 64 CPU output probes per method outside timing. Every application output is checked against a separate compact-CSR CPU reference, which also tests the packing transformation. The expanded test executable also passes Compute Sanitizer memcheck with zero reported errors. Racecheck restricted to the project’s row-owned and pointer-refresh kernels reports zero hazards. Library kernels are outside that racecheck filter. These checks do not substitute for sanitizer coverage of every timed configuration.

4.7 Startup, storage, and repeated products

On native inputs, each method records synchronized host startup time from its prepared host representation through construction, GPU transfer, library initialization or preprocessing, and one completed product. Common input and output buffers and the CUDA context already exist. Matrix storage is constructed separately for each method, and method order is randomized. This measures application-level startup within an existing CUDA process, not cold driver initialization. Correctness work is excluded.

Persistent storage counts the explicitly allocated device arrays, including matrix values and indices, row classifications, pointer and offset arrays, and the workspace requested through cuSPARSE. It excludes common dense panels, host-side metadata, allocator overhead, and opaque library-internal allocations. In particular, grouped cuBLAS storage is not presented as a measurement of its handle’s internal memory use.

For k products, we report the cost model $T_m(k) = S_m + (k - 1)H_m$, where S_m is startup including the first product and H_m is the synchronized host median for subsequent calls. A second model adds measured host geometry and representation assembly. Direct, BSR8, and grouped methods start from packed blocks, while compact CSR and the dense control use the separately timed scalar assembly. Dense expansion is included in its startup observation. The preparation script records one CPU assembly observation per matrix, not a distribution. These are amortization estimates for serial synchronized calls, not timed solver runs, CUDA-graph execution, or asynchronous throughput measurements. For each k , the comparator is the lowest total among all measured library methods. No tuning cost is charged to that lower envelope.

4.8 Aggregation and uncertainty

For configuration c , process label p , comparator b , and direct method d , let

$$y_{c,p} = \log(t_{b,c,p}/t_{d,c,p}), \quad g_c = \exp\left(\frac{1}{3} \sum_{p=1}^3 y_{c,p}\right).$$

The reported geometric mean is $G = \exp(|\mathcal{C}|^{-1} \sum_{c \in \mathcal{C}} \log g_c)$. Each configuration and each process within it has equal weight. Ratios above one favor the direct method. Minima and win counts refer to the configuration ratios g_c , not to individual timing trials.

We also report the range of three aggregate ratios obtained by holding the process label fixed. This is a descriptive repeatability check, not a confidence interval or a test of significance. Three process observations provide limited information about tails and shared thermal drift. No population-level confidence interval is assigned to this deliberately chosen grid, and no comparison of interval widths is interpreted as a decomposition of uncertainty. Matrix-instance variation, workload selection, process noise, and hardware variation remain distinct questions.

5 Results

5.1 Choosing direct execution

Table 1 separates the core grid by panel width. Each row includes all four shape distributions, both localities, and all four degrees. The best-library column takes the fastest measured fixed-address library method within each process, including BSR32 where supported. Its win count therefore asks whether direct execution beats every tested library option for that configuration. The other columns show which alternatives account for the difference.

Table 1: Choosing an implementation by RHS width on the 4,096-block-row core grid. Ratios divide comparator time by direct time. Direct wins and minima use the best-library comparison, averaged over three processes per configuration.

RHS	CSR	Grouped	BSR8	Best library	Direct wins	Min
8	3.513	9.116	1.078	1.057	26/32	0.951
16	4.415	7.778	1.086	1.042	24/32	0.823
32	4.100	4.451	1.128	1.087	21/32	0.903
64	5.190	3.481	1.444	1.397	32/32	1.112

Every panel width has an aggregate advantage over both BSR8 and the best-library envelope. RHS 64 gives the strongest result: 1.444 against BSR8 and wins against every tested library option in all 32 cases. At RHS 8 and 16, direct execution beats BSR8 in all 32 cases per width, with ratios of 1.078 and 1.086, respectively. Uniform blocks remain a useful exception, since BSR32 is preferable in several narrow-panel cases. The width table makes that choice explicit.

Retaining packed blocks also avoids scalar metadata expansion and fixed-tile conversion. Section 5.6 measures the corresponding storage and startup costs. These matter when a matrix is replaced before many products can amortize a library conversion.

5.2 Core synthetic grid

Table 2 compares execution times on the core grid. The direct path has a geometric speedup of 4.262 against the CSR lower envelope and 5.757 against cached grouped cuBLAS. The respective win counts are 128 and 128 out of 128 configurations. Selecting the CSR algorithm reduces the aggregate ratio from 6.834 for default CSR to 4.262 for the lower envelope. The subdivided BSR8 ratio is 1.175, with 124 direct wins out of 128. This comparison retains block structure without padding, comparing two ways to execute the same dense blocks. Figure 3 shows the spread behind these averages. Its sorted curves expose the fastest comparators and the width of the performance range across the grid.

Table 4 breaks down the results by width and degree. Each row averages the four size distributions and two localities. Direct time is the geometric mean of the corresponding process medians. Absolute times distinguish small launch-sensitive cases from larger products. The CSR column uses the lower envelope, and grouped execution uses cached pointers.

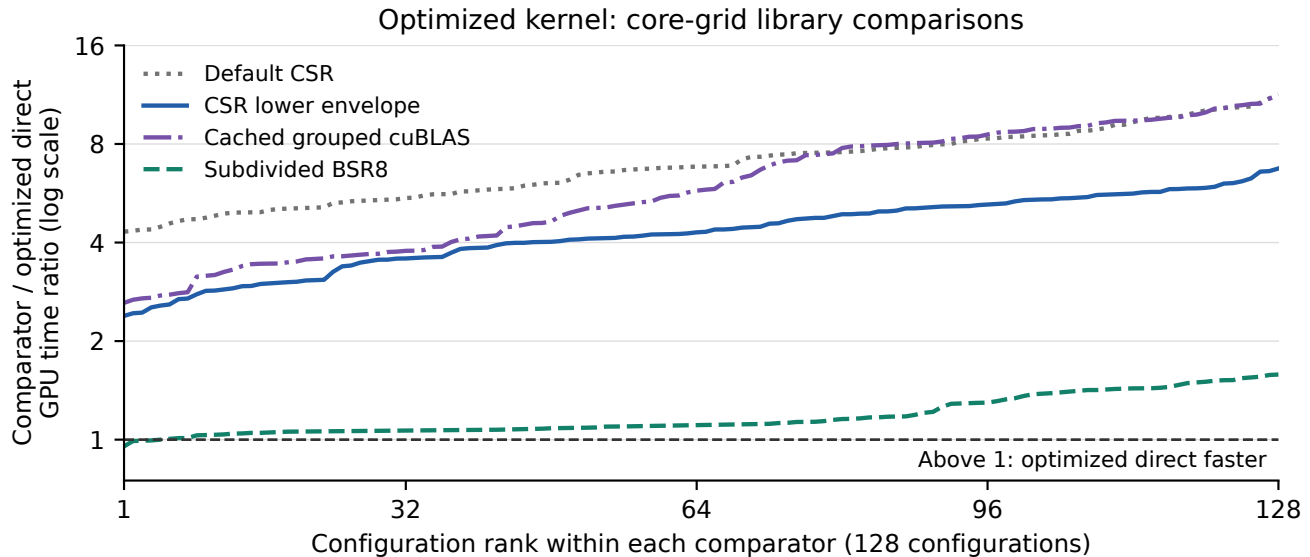


Figure 3: GPU time ratios across the 128 core configurations. Each ratio is the geometric mean of three paired process ratios, computed from per-process timing medians. Each curve sorts configurations independently, so equal ranks need not identify the same matrix. The vertical axis is logarithmic. Ratios above the dashed parity line favor direct execution. These are empirical distributions over the chosen grid, not uncertainty intervals.

Table 2: Core-grid ratios over 128 configurations and three process observations per configuration. Minima and wins use GPU ratios averaged in log space across processes. Changing-address grouped execution is paired with changing-address direct execution.

Comparator	GPU ratio	Host ratio	Minimum	Wins
CSR default	6.834	6.732	4.317	128/128
CSR lower envelope	4.262	4.209	2.387	128/128
Grouped cached	5.757	5.668	2.617	128/128
Grouped changing	5.752	5.661	2.626	128/128
Subdivided BSR8	1.175	1.172	0.954	124/128

Table 3: Descriptive process-label ranges on the core grid. The two endpoints are observed aggregate ratios, not confidence limits.

Comparator	Geometric mean	Process-label range
CSR default	6.834	6.830 to 6.838
CSR lower envelope	4.262	4.253 to 4.268
Grouped cached	5.757	5.755 to 5.759
Grouped changing	5.752	5.749 to 5.754
Subdivided BSR8	1.175	1.174 to 1.176

5.3 Additional matrix seeds and queued execution

The follow-up repeats all 128 core configurations on matrix seeds 2, 3, and 5. Each seed and timing mode uses one fresh process per configuration, giving 768 processes. The two modes synchronize after either one product or eight queued products. Each process measures twenty samples, with GPU and host times divided by the number of products in the sample. The eight products use the same prepared plan and buffers. Changing-address controls alternate the two buffer pairs on each product. Validation runs outside timing, before and after the samples, and outputs are poisoned before each method. Jobs and methods are randomized, and GPU processes run serially.

Table 5 uses the same library lower envelope as the original core comparison. Each row includes 32 configurations on each of three matrix seeds. The seed range contains the smallest and largest 32-configuration geometric means.

Table 4: Width and degree breakdown for the core grid. Times are milliseconds and speedups are dimensionless.

RHS	Degree	Direct ms	CSR ratio	Grouped ratio	BSR8 ratio
8	2	0.114	2.643	7.839	1.105
8	4	0.207	3.667	8.901	1.081
8	8	0.386	5.598	9.741	1.068
8	16	0.751	2.807	10.158	1.058
16	2	0.145	3.553	6.244	1.052
16	4	0.248	4.140	7.577	1.069
16	8	0.447	5.643	8.647	1.109
16	16	0.873	4.576	8.945	1.117
32	2	0.283	3.456	3.461	1.086
32	4	0.508	3.655	4.073	1.039
32	8	0.836	4.476	5.083	1.177
32	16	1.582	4.998	5.480	1.218
64	2	0.415	4.619	2.731	1.445
64	4	0.727	4.995	3.416	1.434
64	8	1.352	5.278	3.825	1.436
64	16	2.588	5.960	4.116	1.461

These ranges describe matrix variation combined with process noise. There is one process for each seed and mode, so they do not separate those sources or define confidence intervals. The original three-process campaign remains separate. The follow-up uses C++20 host compilation and the same library source bytes and CUDA toolkit.

Table 5: Additional matrix seeds and synchronization intervals. Ratios divide comparator time by direct time, so values above one favor direct execution. Batch is the number of products queued before synchronization. Wins compare direct execution with the fastest tested library for each of 96 instances.

Batch	RHS	Best library	BSR8	Wins	Seed range
1	8	1.056	1.081	75/96	1.052 to 1.060
1	16	1.046	1.091	72/96	1.045 to 1.047
1	32	1.091	1.131	65/96	1.087 to 1.094
1	64	1.395	1.443	96/96	1.393 to 1.399
8	8	1.050	1.074	73/96	1.049 to 1.052
8	16	1.034	1.079	72/96	1.032 to 1.036
8	32	1.086	1.125	64/96	1.075 to 1.104
8	64	1.392	1.452	95/96	1.384 to 1.398

Across all 384 instances per mode, the fastest-library ratios are 1.139 for individual products and 1.132 for batches of eight, with 308 and 304 direct wins respectively. Every width has a geometric-mean advantage on each additional seed in both timing modes. RHS 64 gives the largest aggregate advantage, about 1.39, winning all 96 individual-product instances and 95 of 96 queued instances. The narrow-panel gains remain smaller, about three to six percent against the fastest tested library. Queueing eight products preserves this width-dependent pattern on the measured grid.

5.4 Fixed blocks, scaling, and degree imbalance

Both partitions have constant extent in uniform matrices, making fixed-block BSR a suitable control. Across the 32 uniform configurations, the BSR ratio is 0.989, with 10 direct wins. Table 6 shows the width dependence and helps distinguish variable-block reuse from the benefits already available with a uniform block format.

Table 7 compares constant degree and the imbalanced pattern at three sizes. Each row includes both high-variance and bimodal dimensions and all four panel widths. The workload changes affect both total work and load balance. These results therefore test sensitivity to the combined changes and do not isolate the cost of imbalance at a fixed nonzero count. The tested maximum row degree remains 16, so heavy-tailed application degrees are not covered by this synthetic extension.

Table 6: Fixed-block BSR divided by direct execution on the uniform-32 subset. Each width contains eight degree and locality configurations.

RHS	BSR ratio	Minimum	Wins
8	0.982	0.951	2/8
16	0.884	0.823	0/8
32	0.925	0.903	0/8
64	1.192	1.112	8/8

Table 7: Size and degree-pattern controls with random column locality. Each row averages eight configurations and three process observations.

Block rows	Degree pattern	CSR ratio	Grouped ratio	CSR wins
256	16 in every row	2.870	6.428	8/8
256	0, 1, 16, 16	1.492	4.826	8/8
1024	16 in every row	3.056	5.552	8/8
1024	0, 1, 16, 16	3.677	5.294	8/8
4096	16 in every row	4.082	6.224	8/8
4096	0, 1, 16, 16	4.763	5.629	8/8

5.5 Application-derived inputs

Table 8 compares direct execution with compact CSR on the artificially partitioned published matrices. Compact CSR has an aggregate time ratio of 0.352, with 0 direct wins across twelve matrix and width configurations. Direct execution beats cached grouped cuBLAS on the same packed inputs by 5.246 in aggregate. The block-fill column shows how much of the packed storage contains source nonzeros.

Table 8: Compact-CSR lower-envelope time divided by packed direct time for published matrices under the stated artificial partitions. A ratio below one favors compact CSR. Each cell averages three process observations.

Matrix	Block fill	RHS 8	RHS 16	RHS 32	RHS 64
bcsstk13	8.9%	0.223	0.210	0.220	0.311
bcsstk14	12.2%	0.509	0.382	0.548	0.590
bcsstk15	6.9%	0.320	0.315	0.415	0.433

Table 9: All published-matrix controls, averaged across twelve matrix and width configurations. Ratios divide comparator GPU time by direct time.

Comparator	Ratio	Minimum	Direct wins
Compact CSR	0.352	0.210	0/12
BSR8	0.648	0.498	0/12
Grouped cached	5.246	3.361	12/12
Dense SGEMM	1.051	0.460	7/12

Low block fill makes the direct method perform arithmetic on internal zeros that compact CSR omits. These cases test the cost of choosing a packed representation for scalar sparse input. An application should retain compact CSR when its source has scattered nonzeros rather than dense blocks. Figure 4 shows these comparisons alongside uniform BSR, where the original block structure is already fixed.

5.6 Native near-field and lifecycle results

Table 10 describes the geometry-derived partitions. The clustered cases contain variable leaf sizes and much higher degrees than the synthetic grid, while the uniform cases provide small-leaf controls. The table reports overall scalar

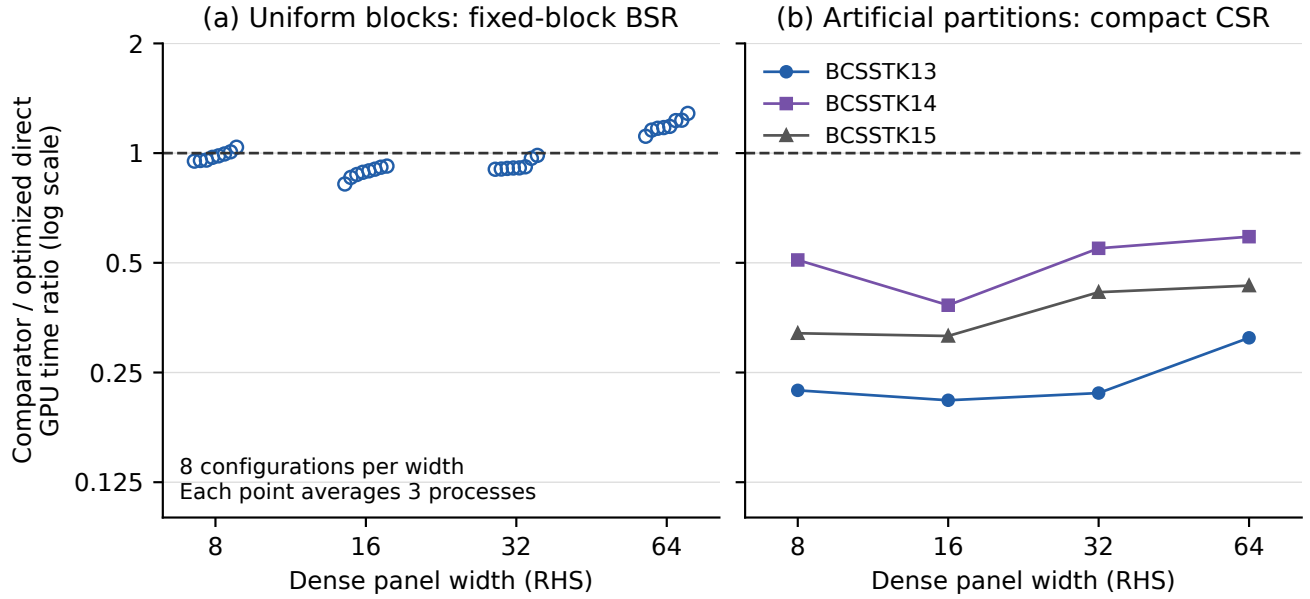


Figure 4: Format controls with a shared logarithmic ratio scale. Each point is the geometric mean of three paired process ratios. Left: all 32 uniform configurations, with eight degree and locality combinations per width. Horizontal offsets separate the points and have no numerical meaning. Right: the compact-CSR lower envelope for the twelve application configurations. Lines connect widths of the same matrix. Ratios below the dashed parity line favor the comparator. The panels represent different workloads and should not be read as a paired comparison between BSR and compact CSR.

density as well as block shape. Dense SGEMM provides a useful alternative for the denser cases. All comparisons compute the identical near-field component.

Table 10: Native-model shape and degree ranges across three geometry seeds.

Points	Geometry	Leaf sizes	Block rows	Mean degree	Density %
2048	uniform	8	256 to 256	122.5 to 127.4	47.8 to 49.8
2048	clustered	8, 16, 32, 64	159 to 167	136.7 to 141.8	88.5 to 89.8
8192	uniform	8, 16	920 to 932	178.0 to 181.7	20.0 to 20.1
8192	clustered	8, 16, 32, 64	454 to 465	219.1 to 226.2	70.1 to 71.1

The aggregate BSR8 ratio on this component is 0.390, so BSR8 is faster for steady-state execution here. Direct execution offers a different benefit in startup, measured below. These matrices have many more blocks per row than the synthetic grid. Their comparison measures the cost of retaining whole block rows under that workload. Geometry, row degree, and leaf size vary together, so the two geometry families do not isolate their individual effects.

Starting from prepared host representations, direct startup has a geometric ratio of 3.953 against the fastest library alternative, including one completed product. The modeled ten-product ratio is 2.668. This measures the adoption cost when packed blocks already exist. The second column of Table 12 adds CPU assembly and therefore answers a different question about constructing the input representations.

The most compact measured scalar-CSR plan uses 1.948 times the explicit device storage of the direct representation, averaged geometrically over native inputs and widths. BSR8 and grouped GEMM retain block storage and have different metadata costs. The host assembly routines are reference implementations and have not been optimized against each other.

Table 11: Near-field comparator GPU time divided by direct time. Each row averages three geometry seeds and three processes per seed. Ratios above one favor direct execution.

Points	Geometry	RHS	CSR	BSR8	Grouped	Dense
2048	uniform	8	0.466	0.371	19.250	0.136
2048	uniform	16	0.557	0.276	13.892	0.121
2048	uniform	32	1.070	0.300	14.079	0.166
2048	uniform	64	0.871	0.252	9.183	0.124
2048	clustered	8	1.012	0.502	16.778	0.163
2048	clustered	16	1.188	0.382	12.269	0.165
2048	clustered	32	1.581	0.286	8.679	0.150
2048	clustered	64	1.754	0.425	7.417	0.129
8192	uniform	8	0.535	0.372	22.483	1.185
8192	uniform	16	0.620	0.253	13.837	1.412
8192	uniform	32	1.032	0.325	11.958	0.490
8192	uniform	64	1.079	0.304	6.529	0.263
8192	clustered	8	1.181	0.598	13.678	0.816
8192	clustered	16	1.836	0.513	10.906	1.256
8192	clustered	32	2.350	0.668	7.097	0.331
8192	clustered	64	3.190	0.836	4.947	0.224

Table 12: Native-input cost models. Ratios divide the best library total by the direct total, averaging equally across input, width, and process. The assembly column adds one recorded CPU preparation observation per matrix and representation.

Products	Prepared-host ratio	Including assembly ratio
1	3.953	0.531
10	2.668	0.544
100	0.824	0.615
1000	0.412	0.491

Table 13: Explicit device-array storage ranges over native inputs and panel widths, and geometric mean synchronized startup time including one product. Storage excludes dense panels and opaque library allocations.

Method	Min MiB	Max MiB	Startup ms
direct	8.02	183.13	6.413
compact_alg1_pre	15.35	364.63	28.311
compact_alg2	15.64	371.50	40.899
compact_alg3_pre	15.33	363.99	65.078
bsr8	7.78	184.79	34.679
grouped_cached	8.85	185.84	37.324
dense	16.00	256.00	194.317

5.7 Matched mechanism results

The 256-thread factorial controls describe the earlier RHS-8 mapping family. The full-panel staging rows compare global and shared input loads with the same 64-thread mapping used by the final narrow kernel. These comparisons separate the contribution of staging from a change in thread count. Per-instance and per-process method medians accompany the tables.

The paired narrow-policy speedups are 1.777 at RHS 8 and 1.771 at RHS 16 across this grid. Both policies use the same input, timing protocol, and full-output reference. Development seed 4 was used to choose the replacement, while this comparison uses seeds 1, 2, and 3. Historical tuning evidence is retained in Appendix A.

Table 14: Matched control time divided by changed-policy time. RHS-8 effects are geometric marginal contrasts over the other two factorial choices. Minima and maxima first average three process ratios for each fixed matrix instance. They are observed ranges, not confidence limits.

RHS	Change	Ratio	Min	Max
8	Eight-row mapping	1.075	0.995	1.153
8	Two accumulators	0.972	0.825	1.084
8	One CTA instead of two	0.943	0.837	1.011
8	Full-panel staging	1.472	1.120	1.966
16	Wider accumulation	1.084	0.918	1.506
16	Cooperative staging	1.406	1.102	1.897
16	Double buffering	1.022	0.925	1.104
16	Full-panel staging	2.271	1.866	2.997
32	Wider accumulation	1.310	1.113	1.598
32	Cooperative staging	1.414	1.205	1.770
32	Double buffering	1.012	0.846	1.191
64	Wider accumulation	1.379	1.217	1.731
64	Cooperative staging	1.632	1.493	1.857
64	Double buffering	0.801	0.661	0.924

Table 15: Previous narrow-policy time divided by final direct time, measured in the same process. Each row includes two degrees, two shape distributions, three matrix seeds, and three processes per seed. Ranges first average the three processes for each matrix instance.

RHS	Block rows	Ratio	Min	Max
8	256	1.565	1.034	1.859
8	1024	2.019	1.677	2.385
16	256	1.846	1.301	2.356
16	1024	1.699	1.392	1.999

6 Scope and limitations

Measurements cover one GPU and one local operating environment. Three process observations describe repeatability over this campaign, while long-term drift and hardware variation require separate measurements. The original synthetic grid uses one matrix seed. The full-core follow-up adds three matrix seeds, with one process per instance and timing mode. The matched and native experiments each use three instances with three processes per instance. These chosen workloads do not define a random application population. Historical measurements use separate protocols and are excluded from the current averages.

The synthetic extension tests one degree-imbalance pattern. The published matrices have artificial partitions, and the covariance component uses eight-point partition granularity to satisfy the block-size contract. Its startup and storage results cover a generated component rather than a complete solver or hierarchical construction pipeline. Total storage and setup still include matrix values and classification metadata.

Library lower envelopes favor the comparators by selecting measured medians without charging a tuning cost. The grouped baseline is the stated slot composition. Compatible research kernels, reordering methods, other block ensembles, and additional precisions are outside this evaluation. The work contributes an implementation and measured execution policy for an established variable-block representation, with performance claims confined to the reported comparisons.

7 Reproducibility

The verified Windows reference workflow uses CUDA 13.4 and MSVC 19.44. The current build uses C++20 for host and device sources. Archived receipts preserve the original compiler settings for earlier measurements. From the repository root, run:

```
python scripts/prepare_application.py
python scripts/prepare_native.py
python scripts/run_final_validation.py --output-root data/rerun
powershell -File scripts/build_paper.ps1 -ResultsRoot data/rerun
```

The final runner builds and tests the source, runs memcheck and filtered racecheck, then collects the five GPU campaigns serially. Individual runners accept fresh output directories for additional campaigns. Run `uv sync --locked` to install the Python 3.13 environment from `pyproject.toml` and `uv.lock`, then activate `.venv` before these commands. Paper building also uses `pdfLaTeX` and `pdftotext`.

`scripts/build_verified.py` hashes all compiled inputs before a clean build, checks them again afterward, runs CTest, and records the resulting executable hashes and build commands. A cached receipt is reused only when every source and executable hash still matches. Each campaign snapshots the source and includes this receipt, and every process records its executable checksum. The analyzers verify the receipt against the source archive and reject missing or mismatched commands, matrix identities, design coverage, method counts, trial indices, nonfinite timings, and unsuccessful processes. The hashes verify internal consistency of the archived artifacts.

Final measurements are under `data/product-evaluation/`, in the `synthetic/`, `published/`, `native/`, and `ablation/` subdirectories, with the additional seeds and queued timings under `data/robustness/`. Generated tables and figures come from these campaigns. Native geometry, partition metadata, assembly observations, and input checksums are retained under `data/native/`. Large native binary arrays are regenerated by the preparation script and verified against their recorded checksums. Regeneration preserves the archived assembly observations. Use a fresh preparation directory to measure assembly again.

`scripts/analyze_revision.py` validates the main library campaigns. `scripts/analyze_supplement.py` validates native and matched controls. Both emit full-precision summaries beside the generated tables. `scripts/build_paper.ps1` requires both analyses to pass before compiling the manuscript. Regression tests in `tests/test_evidence.py` exercise corrupt timing records and stale source or binary receipts. The historical RHS-8 archive is validated separately for the appendix. The two seed-4 development archives are under `data/narrow-tuning/` and `data/narrow-tuning-2/`. Their 96 processes are validated by `scripts/analyze_tuning.py` and are excluded from evaluation averages. It predates build-linked receipts and is not used to generate current library ratios or matched mechanism effects.

8 Conclusion

VarBlockSpMM lets applications retain packed variable blocks while using a small direct CUDA plan. Shared input tiles and multiple accumulators reuse block data, and row ownership removes output atomics and execution workspace. Across the additional matrix seeds, the fastest-tested-library ratios are 1.139 for individual products and 1.132 for batches of eight. The width breakdown identifies where these gains justify choosing the direct implementation.

The 64-thread narrow kernel gives paired speedups of 1.777 at RHS 8 and 1.771 at RHS 16 over the previous policy. Direct execution also avoids library-format conversion and reduces explicit storage relative to scalar CSR on the dense-block component. Those benefits matter when packed blocks are already available or only a few products reuse each matrix. Compact CSR and dense SGEMM remain useful alternatives for scalar sparsity and high overall density, respectively.

A Historical development evidence

This appendix reports archived development measurements. They are separate from the verified-build campaigns in the main text and are not combined with them as additional repetitions or used to adjust current speedups.

A.1 RHS-8 mapping ablation

The RHS-8 ablation covers 144 configurations at 256, 512, 1,024, and 4,096 block rows and columns. At each size, 32 configurations combine degrees 2, 4, 8, 16, the four shape distributions, and both locality patterns. Four

additional configurations combine high-variance or bimodal shapes with both localities and repeating degrees 0, 1, 16, 16.

Each configuration uses matrix seeds 1, 2, and 3, with one fresh process per seed. These are different instances, not repeated processes of one fixed instance. The 432 processes retain 25,920 raw timed calls. Each method receives five warmups and 30 measured calls. Method order is randomized within each process, and jobs are shuffled before serial execution. Input generation, CUDA-event placement, and synchronized host timing use the same definitions as the main campaign. For each configuration, we average the three paired log ratios of GPU medians, then give each configuration equal weight. No significance interval is assigned. The comparator is the scalar RHS-8 kernel, and the tiled method executes the dispatch archived with that campaign, including its small-grid fallback.

Table 16: Scalar baseline time divided by optimized direct time for RHS 8. Each size contains 36 configurations and three matrix seeds. The 256-row paths execute the same scalar kernel, so their deviation from parity reflects measurement variation.

Block rows	RHS-8 mapping	Speedup
256	Original scalar	0.995
512	Eight-row tiles	1.278
1,024	Eight-row tiles	1.491
4,096	Eight-row tiles	2.002

Table 16 shows increasing benefit with grid size. All 108 configurations using the tiled path improve after averaging across seeds, with geometric speedup 1.563 and minimum 1.028. Across all 144 configurations, including the unchanged fallback, the ratio is 1.396. The 4,096-row bimodal/random/degree-16 case improves from 2.245 to 1.086 ms, a ratio of 2.067. These absolute times are geometric means of the three seed medians. The minimum is a descriptive observation, not a statistical guarantee against regression.

Every process passes 64 CPU output probes per method on its timed matrix. A deterministic full-output test exercises all 64 supported block shapes in the tiled path, with empty rows, a non-default stream, and output initialized to NaNs to detect missing writes. That campaign’s updated suite passed memcheck with zero errors and racecheck filtered to the new kernel with zero errors or warnings.

For the representative 4,096-row bimodal/random/degree-16 case, Nsight Compute reports 4,096 CTAs instead of 8,192 and 47 registers per thread instead of 40. Eligible warps per scheduler increase from 0.40 to 0.55, and issued warps per scheduler increase from 0.26 to 0.32. These workload-specific counters support improved scheduling. Instrumented durations do not enter the speedup estimates. The combined experiment does not isolate the contributions of tiling, accumulator reuse, and CTA count. Its paired direct-kernel measurements are separate from the main library comparisons and use three different seeds.

A.2 Earlier width specialization and profiling

The original five-seed dataset remains available under `data/publication/`. It contains 128 configurations per seed, one benchmark process per configuration and seed, five warmups, and 20 measured executions. Its saved summaries reproduce geometric ratios of 4.988 over default CSR and 4.222 over the original grouped composition. That composition rebuilt pointers and cleared all outputs on every call. The original harness also used constant dense input and fixed method order.

These historical values are separate from the main comparison with the cached baseline. Differences between the experiments cannot isolate the effect of one optimization because the software environment, input values, timing protocol, and comparator configuration also changed. The old per-seed standard errors describe variation in five whole-grid averages. They do not quantify process noise or establish which source accounts for most uncertainty. The original configuration bootstrap is a resampling-sensitivity analysis of a chosen grid, not evidence about a randomly sampled application population.

The kernel policy was developed using separate single-seed sweeps with 1,024 block rows, three warmups, and seven measured executions. Table 17 reports those exploratory comparisons. Each ratio compares the named successive variants on the stated slice, with equal configuration weights.

Historical Nsight Compute measurements support the optimization mechanism. An early difficult RHS-64 case had 99.54% achieved occupancy but only 0.48 eligible warps per scheduler. Long-scoreboard waiting accounted for

Table 17: Exploratory development ablations, separate from the main evaluation.

Change	Comparator	Cases	Ratio
Wider accumulation	Four-accumulator direct	128	1.24
Cooperative staging	Wider accumulation	128	1.18
Wider accumulation and staging	Four-accumulator direct	128	1.46
RHS-32 buffering and dispatch	Single-buffer staging	32	1.11

90.1% of its issue interval. A subsequent staging comparison reduced the reported long-scoreboard share from 78.0% to 37.1%, while barrier waiting reached 36.4%. The tall RHS-32 profile confirmed native asynchronous copies. These observations support reduced redundant loading and a shift toward synchronization costs. They are workload-specific counters, and replay-instrumented durations are excluded from speedup calculations.

References

- [1] C. Yang, A. Buluç, and J. D. Owens. Design Principles for Sparse Matrix Multiplication on the GPU. *Euro-Par*, 2018. <https://arxiv.org/abs/1803.08601>
- [2] P. Okanovic et al. High Performance Unstructured SpMM Computation Using Tensor Cores. 2024. <https://arxiv.org/abs/2408.11551>
- [3] NVIDIA. cuSPARSE API reference, SpMM algorithms and preprocessing. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cusparse/index.html#cusparsespm>
- [4] NVIDIA. cuBLAS grouped batched GEMM. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cublas/#cublas-t-gemmgroupedbatched>
- [5] W. Boukaram, Y. Liu, P. Ghysels, and X. S. Li. Adaptive Sketching Based Construction of H^2 Matrices on GPUs. *IPDPSW*, 2025. <https://arxiv.org/abs/2506.16759>
- [6] R. Vuduc and H.-J. Moon. Fast Sparse Matrix-Vector Multiplication by Exploiting Variable Block Structure. *HPCC*, 2005. https://doi.org/10.1007/11557654_91
- [7] Berkeley Benchmarking and Optimization Group. OSKI Variable Block Row Format. 2007. https://bebop.cs.berkeley.edu/oski/html/group__MATTYPE__VBR.html
- [8] G. Huang, G. Dai, Y. Wang, and H. Yang. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. *SC20*, 2020. <https://doi.org/10.1109/SC41405.2020.00076>
- [9] NVIDIA. Asynchronous Data Copies. CUDA Programming Guide. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html>
- [10] SuiteSparse Matrix Collection. HB/BCSSTK13, HB/BCSSTK14, and HB/BCSSTK15. Accessed 5 September 2026. <https://sparse.tamu.edu/HB/bcsstk13>
- [11] NIST Matrix Market. BCS Structural Engineering Matrices. BCSSTK13, BCSSTK14, and BCSSTK15 problem descriptions. <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc1/bcsstk13.html> <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc2/bcsstk14.html> <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc2/bcsstk15.html>